Regularization and hyperparameter optimization

Stefano Diciotti[1]

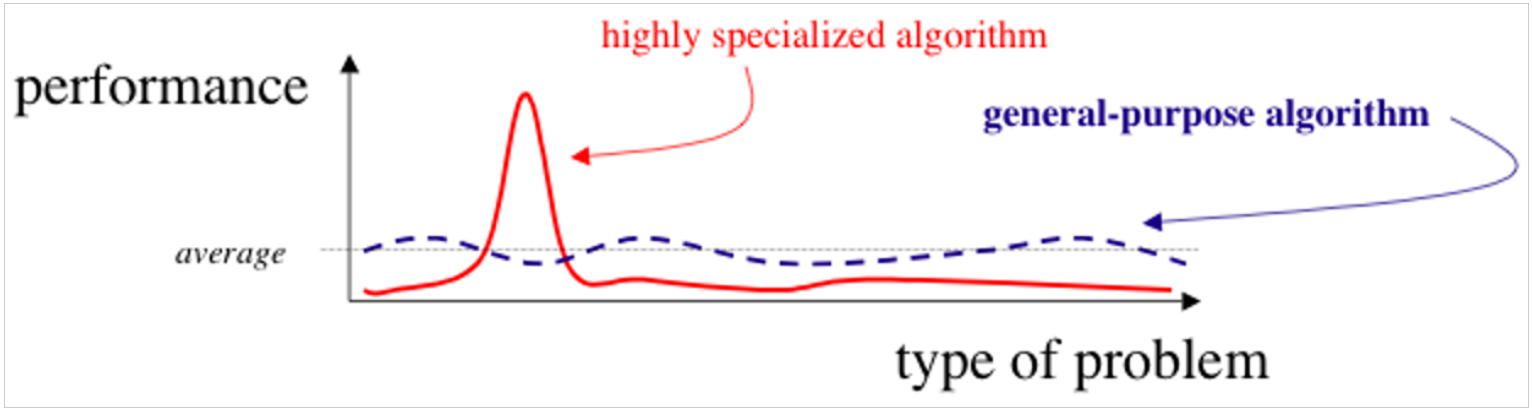
[1] Dipartimento di Ingegneria dell'Energia Elettrica e dell'Informazione "Guglielmo Marconi" - DEI, Università di Bologna

No Free Lunch Theorem (Wolpert, 1996)

No machine learning algorithm is universally any better than any other.

Key point:

- If one algorithm performs better than another on one class of problems, then it will perform worse on another class of problems.



Regularization

The No Free Lunch Theorem implies that we must design machine learning algorithms to perform well on specific tasks.

Two main strategies:

- Modify the hypothesis space:
  - Increase or decrease the model's capacity by adding or removing functions.
- Introduce preferences within the hypothesis space:
  - Favor some solutions over others, unless a less preferred solution fits the data significantly better.

Example:
Modify the training criterion in linear regression by adding **weight decay** (penalizing large weights).

Why Penalize Large Weights?

Penalizing large weights (e.g., via weight decay) improves generalization by:

1. Preventing overly complex models

- Large weights allow the model to fit the training data too closely, including noise and random fluctuations.
- This increases model complexity and leads to **overfitting**: the model performs well on training data but poorly on unseen data.
- **Example:** Even a high-degree polynomial can be simple if its coefficients (weights) are small or close to zero.

2. Reducing model sensitivity

- Large weights amplify small variations in the input, making the model unstable.
- Smaller weights make the output less sensitive to small input perturbations.

3. Promoting more robust solutions

- Small weights ensure that no single feature dominates the prediction.
- The model relies on patterns that are more likely to persist across different datasets, improving robustness and generalization.

Large Weights vs Small Weights

	Large weights	**Small weights**
Model complexity	High (risk of overfitting)	Moderate (better generalization)
Sensitivity to noise	High	Low
Robustness	Poor (unstable predictions)	Good (stable predictions)
Dependence on features	Strong dependence on few features	Balanced reliance on multiple features

Regularization pushes the model toward simpler, more robust solutions.

Linear Regression with Weight Decay

In weight-decayed linear regression, we minimize:

$$J(\mathbf{w}) = \text{MSE}_{\text{train}} + \lambda \mathbf{w}^\top \mathbf{w}$$

Interpretation:

- $J(\mathbf{w})$ expresses a preference for solutions with smaller squared L_2 norm of the weights.

Examples:

- Single feature:

$$\hat{y} = w_1 x_1$$

$$J(w) = \text{MSE}_{\text{train}} + \lambda w_1^2$$

- Two features:

$$\hat{y} = w_1 x_1 + w_2 x_2$$

$$J(w) = \text{MSE}_{\text{train}} + \lambda (w_1^2 + w_2^2)$$

Note:

- λ is a hyperparameter chosen in advance that controls the strength of regularization.

Effect of Weight Decay Strength

Key ideas:

- When $\lambda = 0$, we impose no preference for small weights.
- Increasing λ forces the weights to become smaller.

Minimizing $J(\mathbf{w})$ results in:

- A tradeoff between fitting the training data and keeping the weights small.

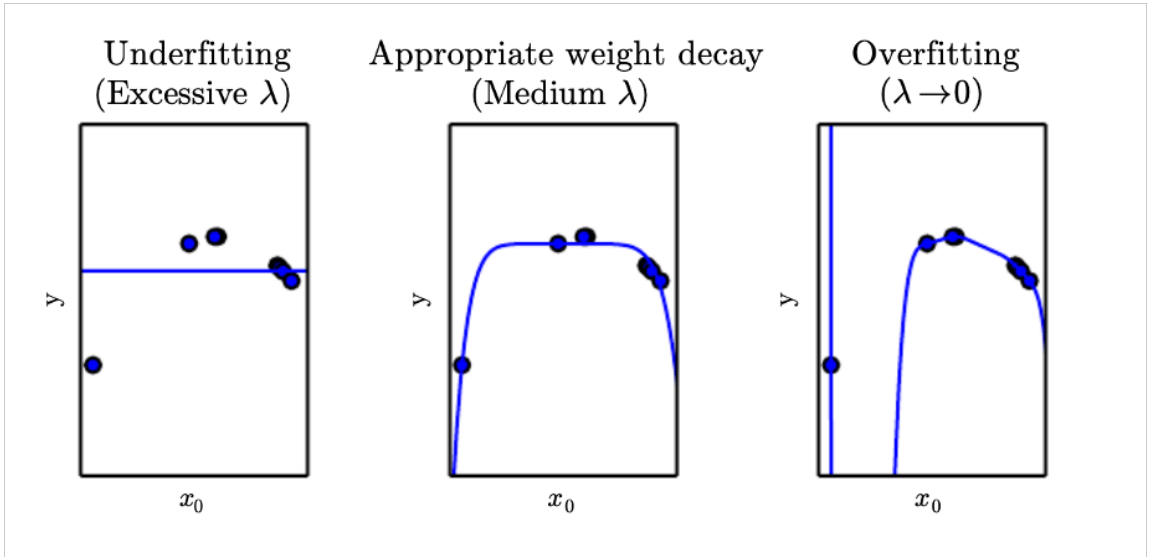


Figure: We fit a high-degree polynomial regression model to our previous example training set

Regularization: Expressing Preferences

Key ideas:

- Expressing preferences for one function over another is a more general way of controlling model capacity than simply including or excluding functions from the hypothesis space.
- There are many ways to express these preferences, collectively known as **regularization**.

Definition:

- Regularization is any modification to a learning algorithm intended to **reduce generalization error** but not its training error.

Hyperparameters and Validation Set

Key ideas:

- Most machine learning algorithms have **hyperparameters** — settings that control the algorithm's behavior.
- Hyperparameters are not learned from the training data: they must be set manually or selected via a validation procedure.

Examples:

- Polynomial regression: the degree of the polynomial is a capacity hyperparameter.
- Weight decay: the regularization strength λ is another example of a hyperparameter.

Hyperparameters and Validation Set

Key ideas:

- It is not appropriate to optimize hyperparameters that control model capacity using the training set.
- If optimized on the training set, hyperparameters would always choose maximum capacity, leading to overfitting.

Example:

- A higher-degree polynomial or setting $\lambda = 0$ (no regularization) always fits the training data better — but at the cost of poor generalization.

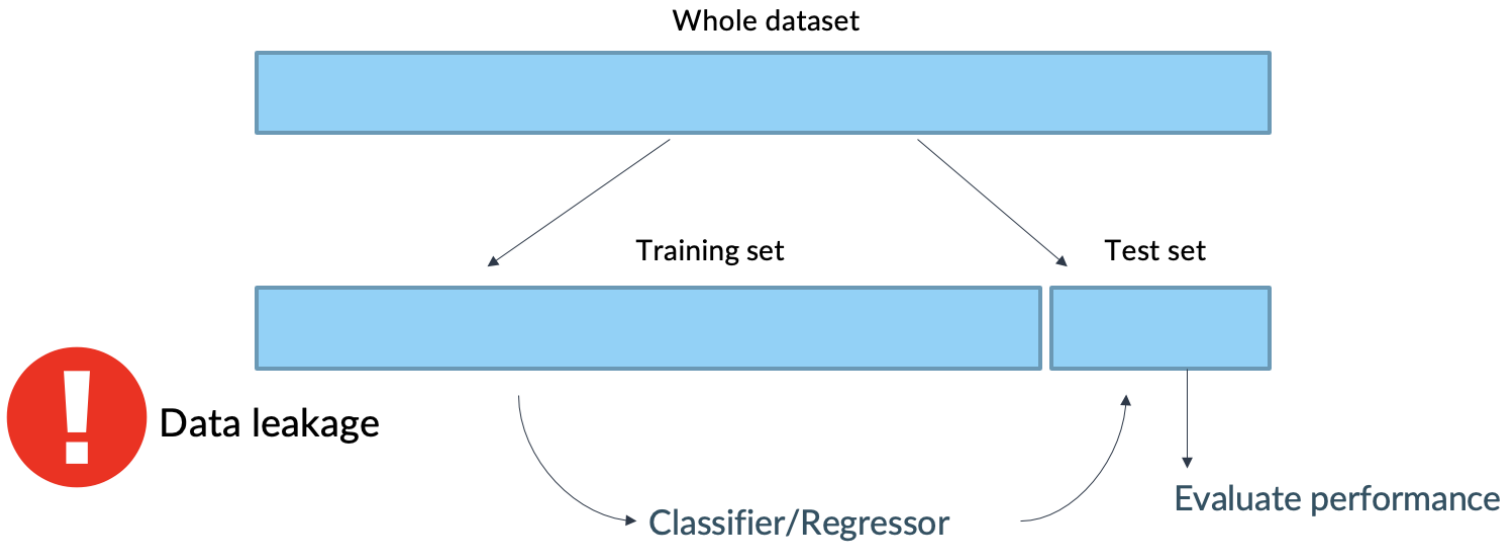
Solution:

- Use a **validation set** — a separate set of examples not seen by the training algorithm — to select hyperparameters.

Hyperparameters and Validation Set

Hold-out validation scheme:

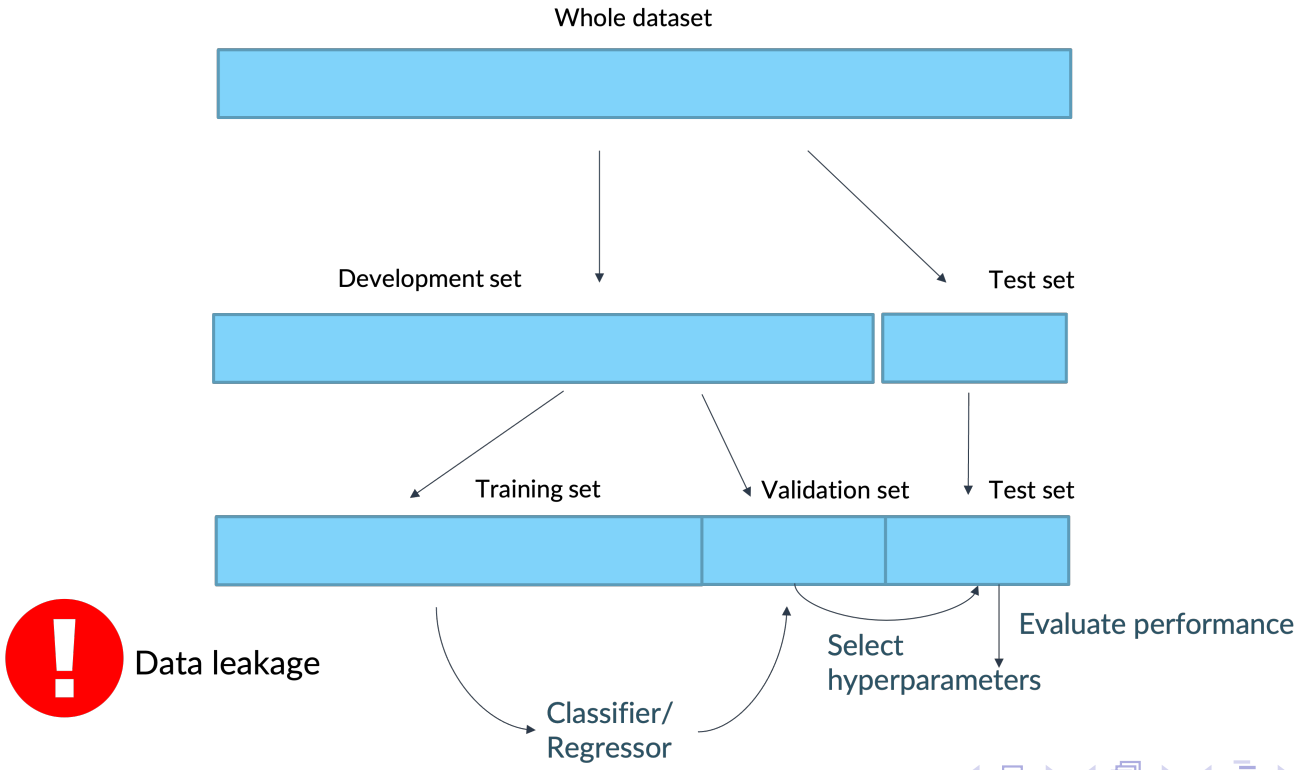
- The dataset is randomly split into a training set and a test set.
- The test set is composed of examples drawn from the same distribution as the training set.
- Test instances must never be used to make choices about the model, including selecting hyperparameters.
- Simple hold-out validation is not sufficient for hyperparameter optimization: using the test set to tune hyperparameters leads to optimistic bias.



Hyperparameters and Validation Set

Nested hold-out validation scheme:

- The dataset is randomly split into two disjoint parts:
  - Development set: used for training the model and tuning hyperparameters.
  - Test set: used only for the final evaluation.
- Then, the development set is further randomly split into:
  - Training set: used to learn the model parameters.
  - Validation set: used to estimate generalization error and adjust hyperparameters accordingly.
- This procedure ensures that hyperparameter optimization does not bias the test set evaluation.



Stratified Hold-Out and Nested Hold-Out

Stratified hold-out:

- Instead of splitting the data randomly, we split while preserving the class distribution across the training and test sets.
- Important especially with imbalanced datasets.
- Ensures that both training and test sets are representative of the original dataset's proportions.
- **Example:** If the original dataset has 30% positives and 70% negatives, both the training and test sets will maintain approximately the same 30/70 ratio.

Stratified nested hold-Out:

- Both the outer split and the inner split are performed in a stratified manner.
- Preserves class balance during:
  - Hyperparameter tuning (inner split)
  - Final evaluation (outer split)
- **Example:** During the inner hold-out to select hyperparameters, we ensure that the validation subset has the same proportion of each class as the training subset.

Nested Hold-Out: Model Comparison and Feature Selection

Applications of nested hold-out beyond hyperparameter optimization:

- Model comparison:
  - Use the inner hold-out split to select the best model among several candidates (e.g., SVM, random forest, artificial neural network).
  - Use the outer hold-out set to evaluate the generalization performance of the selected model.
- Feature selection:
  - Use the inner hold-out split to select the best subset of features.
  - Use the hold-out set to estimate the generalization error after feature selection.

Key idea:

- The inner split is used for making choices (hyperparameter tuning, model selection, feature selection).
- The outer split is used only for an unbiased evaluation of the final decision.

Hyperparameters and Validation Set

Validation set bias:

- Since the validation set is used to “train” the hyperparameters, its error tends to underestimate the true generalization error, although typically by less than the training error would.

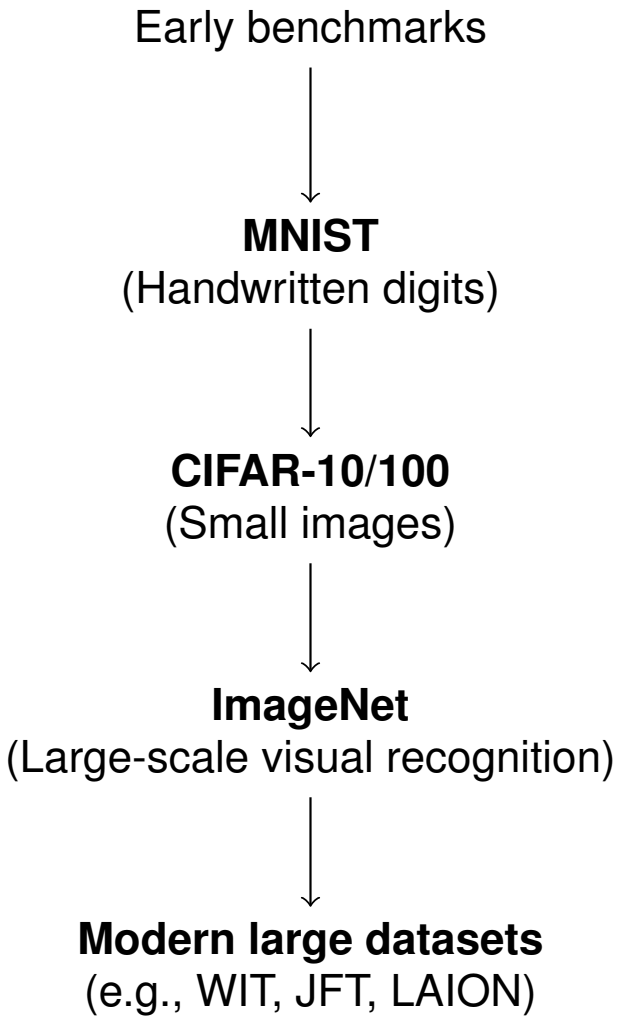
After hyperparameter tuning:

- The generalization error should be estimated only on the test set.

In practice:

- Repeated reuse of the same test set over many years can lead to optimistic bias — test results no longer reflect true field performance.
- The community responds by moving to new and larger benchmark datasets.

Evolution of Benchmark Datasets (Non-biomedical examples)



- As benchmarks become saturated, new datasets are adopted to better evaluate true generalization.
- These are examples from general computer vision, not from the biomedical field.

References

- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, Chapter 5
  `https://www.deeplearningbook.org/`